

# Sorting and Patterns - Day 2

## Why best-case time differs

Priyanka's Platinum study plan | Tuesday, September 8, 2026 | 120 minutes | C11 and Python 3

Today's question: which input arrangements actually remove work from our code? This session is devoted to analysis problems, instrumentation, and one sliding-window application. Reuse yesterday's functions. Begin in C today, then translate to Python. Prerequisites: the three sorting loops and basic sums.

### Your two-hour session

| Minutes | Task                                     | Deliverable                            |
|---------|------------------------------------------|----------------------------------------|
| 0-15    | Define the cost model and cases          | Precise meanings of best/average/worst |
| 15-40   | Lab A: exact comparison counts           | Six counts with loop-bound sums        |
| 40-60   | Lab B: inversions and bubble direction   | Two contrasting input families         |
| 60-80   | Instrument both languages                | Predictions checked against counters   |
| 80-105  | Lab C: aggregate loops and fixed windows | One proof and two implementations      |
| 105-120 | Measure runtime; exit ticket             | Ratios, limitations, corrections       |

Attempt the analysis before running code. The appendix is a checking tool, not the starting point. Keep each block timeboxed: if a function is unfinished, use its reference to identify and fix the first disagreement.

### What are we measuring?

Let  $T(A)$  count a specified operation on input  $A$ . For inputs of length  $n$ , best time is the minimum  $T(A)$ , worst time the maximum, and average time an expectation under a stated distribution.  $O$  is an upper bound,  $\Omega$  a lower bound, and  $\Theta$  a tight bound. They are not synonyms for worst, best, and average. For example, selection sort's best-case time is  $\Theta(n^2)$ .

Count key comparisons such as  $a[j] > \text{key}$  separately from loop-condition checks, swaps, shifts, and allocations. Count a swap as one event, not three assignments; count an insertion shift separately from the final key write. Python tuple assignment and C temporary-variable swaps perform the same conceptual swap without promising equal measured time.

For ordinary complexity here, use unit-cost arithmetic on bounded integers. Python supports arbitrary-sized integers; if bit lengths grow, arithmetic cost must be included. For C counters and window sums, use unsigned long long and long long respectively and limit exercise sizes so these do not overflow ( $n \leq 1000000$  with 32-bit array values is ample for these exercises).

### Lab A - Explain each term, then sum

For  $n=5$ , use  $S=[1,2,3,4,5]$  and  $R=[5,4,3,2,1]$ . Fill a table with comparisons, swaps or shifts, and final-key writes for insertion. Generalize the counts to  $n \geq 2$  using Day 1's exact implementations.

- Selection's inner loop scans lengths  $n-1, n-2, \dots, 1$ . Derive the total on  $S$  and  $R$ . Why does discovering that the current element is smallest not prove the rest of the array is sorted?
- Bubble with early exit: how many comparisons prove  $S$  is sorted? How many on  $R$ ? Remove early exit: what changes in the best case?
- Insertion: how many successful 'larger than key' comparisons occur on  $R$ ? On  $S$ , why is there one failed key comparison per insertion rather than zero comparisons?
- On uniformly random permutations of distinct keys, a given pair is inverted with probability  $1/2$ . Derive the expected number of inversions. Which sorting operations can you relate directly to it?

5. All keys equal: which comparisons and early exits occur? Would changing  $>$  to  $>=$  preserve stability or adaptivity?

## Lab B - Nearly sorted is not a complete explanation

An inversion is a pair of indices  $i < j$  with  $a[i] > a[j]$ . Count inversions in  $[1, 2, 3, 5, 4, 6]$ . Now count them in  $A = [6, 1, 2, 3, 4, 5]$  and  $B = [2, 3, 4, 5, 6, 1]$ .

For insertion sort, each right shift crosses exactly one inverted pair. Explain why shifts equal  $I$ , the initial inversion count, and why total time is  $\Theta(n+I)$ , including the outer-loop overhead and unsuccessful final comparisons.

For left-to-right bubble sort, trace complete passes on  $A$  and  $B$ . A large value can travel all the way right in one pass, but a small value moves left only one position per pass. Both arrays have  $I = n-1$  inversions. Does equal swap count imply equal comparison count? Generalize to  $A = [n, 1, \dots, n-1]$  and  $B = [2, \dots, n, 1]$ .

Instrumentation task: add counters to the three sorts in C and Python. Run  $n=4, 5, 6$  on  $S, R, A, B$  and all-equal inputs. Print actual and predicted counts. Count a key comparison immediately before evaluating it, including the comparison that fails and ends an insertion scan. Do not count an unexecuted comparison after  $j$  reaches zero.

## Lab C - Nested syntax versus total work

Analyze these loops for  $n \geq 1$ . Give a tight bound and a reason, not just a label.

TEXT

```
A: for i in 0..n-1: for j in 0..i-1: visit(i,j)
B: j=0; for i in 0..n-1: while j<n: j+=1
C: x=1; while x<n: x*=2
D: for i in 0..n-1: j=0; while j<n: j+=1
```

Now solve: given integer array  $a$  and  $k$ , return the maximum sum of exactly  $k$  consecutive values. Require  $1 \leq k \leq n$ . Python raises `ValueError` for invalid  $k$ ; C returns false and leaves output untouched. Example  $[-5, -2, -7]$ ,  $k=2 \rightarrow -7$ . A result of zero is incorrect: an empty window is not allowed.

Naive approach: sum each window from scratch, doing  $k(n-k+1)$  additions. New state: the exact sum of the current  $k$ -element interval. Initialize the first window, then subtract the outgoing value and add the incoming value. After processing right, total equals the sum of  $a[\text{right}-k+1:\text{right}+1]$ . Set best to the first real window, not zero. Implement both languages without copying slices.

Tests:  $[3, -1, 4, -2, 5]$ ,  $k=3 \rightarrow 7$ ;  $[7]$ ,  $k=1 \rightarrow 7$ ; all-negative input;  $k=n$ ;  $k=0$ ;  $k>n$ ; empty input. Explain  $O(n)$  time and  $O(1)$  extra space. The naive formula can be  $O(n)$  when  $k$  is fixed or  $k=n$ , and  $\Theta(n^2)$  when  $k$  is proportional to  $n$  away from the endpoints; do not call every choice of  $k$  quadratic.

## Runtime experiment and exit ticket

Use uninstrumented sorts: counters add overhead. For each language separately, time  $n=128, 256, 512$  on sorted, reversed, and seeded random inputs. Prepare a fresh copy before EACH timed sort; otherwise the first run changes later inputs. Exclude generation, copying, printing, and correctness checks. Repeat five times and report the median using Python `time.perf_counter` or C `clock/CLOCKS_PER_SEC`. If C timings round to zero, time a batch of pre-copied arrays and report per-sort time.

Predict roughly 2x growth for linear work and 4x for quadratic work when doubling  $n$ ; small samples can deviate through timing noise, interpreter overhead, branch behavior, and memory effects. Compare growth within a language rather than attributing a C/Python speed difference entirely to the algorithm.

Exit: explain why identical best-case input does not give identical best-case time. Explain why two nested loops can be linear. Submit count tables, both instrumented implementations, fixed-window code, and runtime ratios with one limitation.

# Solution appendix

## Exact counts and derivations

Selection comparisons are  $n(n-1)/2$  on every input: 10 at  $n=5$ . With the conditional swap, S needs 0 swaps, R needs  $\text{floor}(n/2)=2$  swaps at  $n=5$ . Bubble on S uses  $n-1=4$  comparisons and no swaps; on R it uses 10 comparisons and 10 swaps. Without early exit, bubble uses  $n(n-1)/2$  comparisons even on sorted input.

Insertion on S makes  $n-1$  comparisons, 0 shifts,  $n-1$  final-key writes. On R it makes  $n(n-1)/2$  comparisons and shifts, plus  $n-1$  final-key writes. On random distinct permutations, expected inversions are  $n(n-1)/4$ , by summing the  $1/2$  probability over all pairs. Insertion shifts and bubble adjacent swaps each equal  $I$ . Selection swaps do not equal  $I$ .

All-equal input behaves like sorted input under strict comparisons. Insertion with  $\geq$  shifts equal keys and becomes quadratic on this input; bubble with  $\geq$  also swaps equal keys and defeats early exit. Both lose stability.

## The asymmetric bubble examples

[1,2,3,5,4,6] has  $I=1$ . Both A and B have  $I=5$  when  $n=6$ . A becomes sorted after the first bubble pass, then uses a no-swap verification pass:  $5+4=9$  comparisons. B needs  $5+4+3+2+1=15$ . In general A uses  $2n-3$  comparisons for  $n \geq 3$ , while B uses  $n(n-1)/2$ . Both use  $n-1$  swaps. Thus insertion can be analyzed as  $\Theta(n+I)$ , but these bubble comparisons cannot be bounded by that same expression on all inputs.

## Aggregate analysis answers

A sums  $i$  over  $n$  outer iterations:  $\Theta(n^2)$ . B has  $n$  outer iterations and only  $n$  total increments of  $j$ :  $\Theta(n)$ . C takes  $\text{ceil}(\log_2 n)$  iterations: logarithmic growth, with zero iterations at  $n=1$ . D resets  $j$  each time and does  $n*n$  increments:  $\Theta(n^2)$ . The placement of initialization matters more than indentation alone.

The fixed window is initialized in  $k$  steps, then shifted  $n-k$  times. This gives  $\Theta(n)$  time for valid  $k$  and  $O(1)$  workspace. For [3,-1,4,-2,5], window sums are 6,1,7. Negative values cause no problem for a FIXED window; tomorrow's variable sum window needs positivity.

## Hints and repair

If a count disagrees, use  $n=3$  and label every executed key comparison. If a window answer is too high on negative data, inspect best's initial value. If a timing is suspiciously fast, verify the input was not sorted by a previous run. The reference code below uses method 0/1/2 in C and insertion/bubble/selection strings in Python; other C method values are outside the exercise contract.

## Python 3 reference functions

PY

```
def sort_counts(a, method):
    comparisons = moves = 0
    if method == "insertion":
        for i in range(1, len(a)):
            key, j = a[i], i
            while j > 0:
                comparisons += 1
                if a[j - 1] <= key:
                    break
                a[j] = a[j - 1]
                moves += 1 # shifts; excludes the final key write
                j -= 1
            a[j] = key
    elif method == "bubble":
        for end in range(len(a) - 1, 0, -1):
            changed = False
            for j in range(end):
                comparisons += 1
                if a[j] > a[j + 1]:
                    a[j], a[j + 1] = a[j + 1], a[j]
                    moves += 1 # swaps, not individual assignments
                    changed = True
            if not changed:
                break
    elif method == "selection":
        for i in range(len(a) - 1):
            smallest = i
            for j in range(i + 1, len(a)):
                comparisons += 1
                if a[j] < a[smallest]:
                    smallest = j
            if smallest != i:
                a[i], a[smallest] = a[smallest], a[i]
                moves += 1
    else:
        raise ValueError("unknown sort")
    return comparisons, moves
```

PY

```
def max_window(a, k):
    if not 1 <= k <= len(a):
        raise ValueError("require 1 <= k <= n")
    total = 0
    for i in range(k):
        total += a[i]
    best = total
    for right in range(k, len(a)):
        total += a[right] - a[right - k]
        best = max(best, total)
    return best
```

## C11 reference functions

C

```
#include <stdbool.h>
#include <stddef.h>
```

C

```
typedef struct { unsigned long long comparisons, moves; } Counts;
```

C

```
/* method: 0=insertion, 1=bubble, 2=selection. */
Counts sort_counts(int *a, size_t n, int method) {
    Counts c = {0, 0};
    if (method == 0) {
        for (size_t i = 1; i < n; ++i) {
            int key = a[i]; size_t j = i;
            while (j > 0) {
                ++c.comparisons;
                if (a[j - 1] <= key) break;
                a[j] = a[j - 1]; ++c.moves; --j;
            }
            a[j] = key;
        }
    } else if (method == 1) {
        for (size_t end = n; end > 1; --end) {
            bool changed = false;
            for (size_t j = 0; j + 1 < end; ++j) {
                ++c.comparisons;
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                    ++c.moves; changed = true;
                }
            }
            if (!changed) break;
        }
    } else if (method == 2) {
        for (size_t i = 0; i + 1 < n; ++i) {
            size_t smallest = i;
            for (size_t j = i + 1; j < n; ++j) {
                ++c.comparisons;
                if (a[j] < a[smallest]) smallest = j;
            }
            if (smallest != i) {
                int t = a[i]; a[i] = a[smallest]; a[smallest] = t;
                ++c.moves;
            }
        }
    }
    return c;
}
```

C

```
bool max_window(const int *a, size_t n, size_t k, long long *out) {
    if (k == 0 || k > n) return false;
    long long total = 0;
    for (size_t i = 0; i < k; ++i) total += a[i];
    long long best = total;
    for (size_t right = k; right < n; ++right) {
        total += (long long)a[right] - a[right - k];
        if (total > best) best = total;
    }
    *out = best;
    return true;
}
```